

TD — Continuations et effets

Réponses uniquement

1. Continuation-Passing Style

Exercice 1

```
let height_cps tree =
  let rec aux t k =
    match t with
    | Leaf _ -> k 0
    | Node (l, r) ->
      aux l (fun hl ->
        aux r (fun hr ->
          k (1 + max hl hr)))
  in
  aux tree (fun x -> x)
```

2. Effets — Rattrapage d'erreurs

Exercice 2

```
val bad_name : unit -> string
val not_int : string -> int
```

Exercice 3

```
effect Bad_name : unit -> string
effect Not_int : string -> int
```

```
let bad_name () = perform (Bad_name ())
let not_int s = perform (Not_int s)
```

Exercice 4

```
let run () =
  let rec handler f =
    try f () with
    | effect (Bad_name ()) k ->
      let n = read_line () in
      continue k n
    | effect (Not_int _) k ->
      let s = read_line () in
      continue k (int_of_string s)
  in
  handler (fun () -> proc (read_line ()))
```

3. Itérateurs et flux

Exercice 5

```
let rec iter_tree f t =
  match t with
  | Leaf x -> f x
  | Node (l, r) ->
    iter_tree f l;
    iter_tree f r
```

Exercice 6

```
let rec equal (Tick f1) (Tick f2) =
  match f1 (), f2 () with
  | None, None -> true
  | Some (x1, r1), Some (x2, r2) ->
    x1 = x2 && equal r1 r2
  | _ -> false
```

Exercice 7

```
let flux_of_iter iter =
  let q = Queue.create () in
  let finished = ref false in
  iter (fun x -> Queue.add x q);
```

```
let rec next () =
  if Queue.is_empty q then
    if !finished then None
    else (finished := true; next ())
  else
    let x = Queue.take q in
    Some (x, Tick next)
in
Tick next
```

Exercice 8

```
let same_border t1 t2 =
  let f1 = flux_of_iter (fun f -> iter_tree f t1) in
  let f2 = flux_of_iter (fun f -> iter_tree f t2) in
  equal f1 f2
```

4. Green Threads

Exercice 9

```
let ping () =
  for i = 1 to 10 do
    print_endline "ping !";
    yield ()
  done
```

```
let pong () =
  for i = 1 to 10 do
    print_endline "pong !";
    yield ()
  done
```

```
let main () =
  spawn ping;
  spawn pong
```

Exercice 10

```
effect Yield : unit
effect Spawn : prog -> unit
```

```
let yield () = perform Yield
let spawn p = perform (Spawn p)
```

Exercice 11

```
let scheduler main =
  let q = Queue.create () in
  let rec run p =
    try p () with
    | effect Yield k ->
      Queue.add (fun () -> continue k ()) q;
      next ()
    | effect (Spawn p') k ->
      Queue.add (fun () -> continue k ()) q;
      run p'
  and next () =
    match Queue.take_opt q with
    | None -> ()
    | Some p -> run p
  in
  run main
```

Continuations et effets

1 Continuation-passing style

On donne le type des arbres binaires suivant :

```
type 'a tree = Leaf of 'a | Node of 'a tree * 'a tree
```

La fonction qui calcule la hauteur de l'arbre peut s'écrire :

```
let rec height tree = match tree with
| Leaf _ -> 0
| Node (l, r) -> 1 + max (height l) (height r)
```

Cette version de `height` n'est pas récursive terminale, et l'ajout d'un simple accumulateur ne permettrait pas qu'elle le devienne. En revanche, comme vu en cours, la transformation CPS permet d'obtenir systématiquement une fonction récursive terminale.

■ **Exercice 1:** Appliquer la transformation CPS pour obtenir une version récursive terminale de `height`.

2 Effets : rattrapage d'erreurs

On souhaite écrire un programme qui lit un entier situé sur la première ligne d'un fichier dont le nom est passé en paramètre. Si un fichier de ce nom n'existe pas, le nom doit être redemandé interactivement à l'utilisateur. Si le fichier existe et que la première ligne n'est pas un entier, un entier doit être demandé à l'utilisateur.

```
let ouvre nom =
  open_in (if Sys.file_exists nom then nom else bad_name ())
```

```
let lit_ligne fic =
  let s = input_line fic in
  match int_of_string_opt s with
  | None -> not_int s
  | Some r -> r
```

```
let proc nom =
  Format.printf "tentative de lecture du fichier '%s@.'" nom;
  let f = ouvre nom in
  let i = lit_ligne f in
  close_in f;
  Format.printf "Entier lu : %d@." i
```

■ **Exercice 2:** Donner le type des opérations `bad_name` et `not_int`.

■ **Exercice 3:** Implémenter ces opérations.

■ **Exercice 4:** Écrire une fonction `run : unit -> unit` qui appelle `proc` et gère les effets qu'elle produit.

3 Enumérer à partir d'un itérateur

On rappelle qu'un itérateur sur un conteneur (ex : liste, arbre...) est une fonction d'ordre supérieur qui prend une fonction `elt -> unit` en paramètre (où `elt` est le type des éléments) et l'applique une fois à chaque élément, dans un ordre déterminé.

■ **Exercice 5:** Implémenter l'itérateur sur le type des arbres binaires polymorphes.

Le *problème des bordures identiques* consiste à déterminer si deux arbres ont la même bordure, c'est-à-dire les mêmes feuilles lues de gauche à droite. Par exemple, les deux arbres suivants ont la même bordure :



Une façon simple de résoudre ce problème est de prendre le flux des éléments des deux arbres et de vérifier qu'ils sont égaux.

■ **Exercice 6:** Soit le type des flux : `type 'a flux = Tick of (unit -> ('a * 'a flux) option)`. Écrire la fonction `equal : 'a flux -> 'a flux -> bool` qui exprime l'égalité des flux point-à-point.

■ **Exercice 7:** Écrire une fonction `flux_of_iter` qui, étant donné un itérateur sur *n'importe quel conteneur d'entiers*, renvoie le flux de ces entiers.

■ **Exercice 8:** Résoudre le problème des bordures identiques.

4 Green threads

Une possibilité intéressante en programmation est de permettre la création et manipulation de *threads*, i.e. de processus légers, au niveau utilisateur, comme une librairie standard. Cette solution évite les appels systèmes de gestion de processus, réputés lourds, et rend de fait la notion de processus relativement indépendante du système. Ces processus utilisateurs, plus économes en ressources système, sont appelés *green threads*.

On suppose disponibles les deux fonctions suivantes :

```
type prog = unit -> unit
```

```
val yield : unit -> unit
(** quand le processus courant appelle cette fonction, il rend la main *)
```

```
val spawn : prog -> unit
(** [spawn p] rend la main et exécute [p] *)
```

■ **Exercice 9:** Modifiez le programme suivant pour que `ping` et `pong` puissent être exécutés de façon concurrente :

```
let ping () =
  for i = 1 to 10 do
    print_endline "ping !"
  done
let pong () =
  for i = 1 to 10 do
    print_endline "pong !"
  done
let main () = ping (); pong ()
```

■ **Exercice 10:** Implémenter `yield` et `spawn`.

■ **Exercice 11:** Définir le *handler* implémentant pour cet effet le passage d'un *thread* au suivant à chaque fois que le premier rend la main. On peut utiliser le module `Queue` pour stocker les programmes mis en pause, dont voici quelques fonctions :

```
val create : unit -> 'a t
Return a new queue, initially empty
```

```
val add : 'a -> 'a t -> unit
[add x q] adds the element [x] at the end of the queue [q].
```

```
val take_opt : 'a t -> 'a option
[take_opt q] removes and returns the first element in queue [q], or returns [None] if the queue is empty.
```